🔥 DAY 17 — CACHING (SPEED + COST OPTIMIZATION)

GOAL

Upgrade from:Repeat work every time ❌

to: Reuse previous results ✅

WHAT YOU CACHE

1. LLM responses

2. Retrieval results

3. Embeddings (optional later)

✅ STEP 1 — CREATE CACHE FILE

📄 utils/cache.py
📍 Create new folder if not exists: utils/

✅ Code

```
import json

import os

import hashlib

class Cache:

  def __init__(self, path="cache/cache.json"):

    self.path = path

    os.makedirs(os.path.dirname(self.path), exist_ok=True)

    if not os.path.exists(self.path):

      with open(self.path, "w") as f:

        json.dump({}, f)

  def _get_key(self, text):

    return hashlib.md5(text.encode()).hexdigest()

  def get(self, text):
```

```
        key = self._get_key(text)
        with open(self.path, "r") as f:
            data = json.load(f)
        return data.get(key, None)
    def set(self, text, value):
        key = self._get_key(text)
        with open(self.path, "r") as f:
            data = json.load(f)
        data[key] = value
        with open(self.path, "w") as f:
            json.dump(data, f, indent=2)
```

✅ **STEP 2 — USE CACHE IN LLM**

📄 models/llm.py

✅ **MODIFY FILE**

👉 Add import: from utils.cache import Cache

✅ **CREATE CACHE INSTANCE**

```
cache = Cache()
```

✅ **UPDATE generate()**

```
def generate(prompt):
    # ✅ Check cache first
    cached = cache.get(prompt)
    if cached:
        print("[CACHE HIT]")
        return cached
```

```
    response = ollama.chat(
        model="qwen:latest",
        messages=[{"role": "user", "content": prompt}]    )
    result = response["message"]["content"]
    # ✅ Save to cache
    cache.set(prompt, result)
    return result
```

✅ **STEP 3 — CACHE RAG SEARCH (OPTIONAL BUT IMPORTANT)**

📄 retrieval/search.py (or your HybridSearch file)

✅ **MODIFY search()**

```
from utils.cache import Cache
class HybridSearch:
    def init(self):
        self.cache = Cache()
    def search(self, query):
        # ✅ Cache check
        cached = self.cache.get("RAG:"+query)
        if cached:
            print("[CACHE HIT - RAG]")
            return cached
        # existing logic
        results = ...  # your current retrieval
        # ✅ store cache
        self.cache.set("RAG:"+query, results)
        return results
```

✅ **STEP 4 — CREATE CACHE FOLDER**

mkdir cache

✅ **FINAL FLOW NOW**

User Query

↓

Cache Check ✅

↓

If HIT → return instantly

↓

If MISS → run system

↓

Store result in cache

✅ **TEST**

Run: uvicorn app.api:app --reload

✅ **Try SAME query twice:**

>> what is ai

>> what is ai

✅ **EXPECTED OUTPUT**

First time → normal

Second time → [CACHE HIT]

✅ **WHAT YOU BUILT TODAY**

Caching system

✅ **SYSTEM LEVEL NOW**

You now have:

✅ AI system

✅ API + Docker

✅ Async execution

✅ ✅ Caching (NEW)

✅ **BENEFITS**

⚡ Faster response

💰 Lower cost

🔁 Avoid recomputation

✅ **INDUSTRY TERM**

Response Caching Layer

Used in:

- OpenAI systems
- Copilot
- Production LLM pipelines

✅ **NEXT**

DAY 18 → Prompt Optimization (make answers smarter)

If ready:
👉 say **Day 18**